

NESNE YÖNELİMLİ PROGRAMLAMA

Nesne Yönelimli Programlama Paradigması

Python, sadece popüler bir **betik** (**script**) dili olarak değil, aynı zamanda nesne yönelimli programlama paradigmasını da destekler. Bu paradigmada **nesneler** (**object**) programın temel yapıtaşlarıdır. Nesneler; **durum** (**state**) ve **davranışlara** (**behavior**) sahiptir. Programlama, nesnelerin birbirlerine **ileti göndermesiyle** (**message-passing**) yapılır. Emreden paradigmanın aksine bu paradigmada programlama, nesnelerin birbirlerine ileti göndermesi ile yapılır.

Nesne yönelimli programlama paradigması aşağıdaki kavramlar üzerine kuruludur;

1. **Sınıf** (class): Nasıl ki dünyamızda evler bir **mimari plan** (**blueprint**) üzerinden inşa ediliyor, **nesneler** (**object**) de bir plan üzerinden inşa edilir. Nesnelere ilişkin verilerin tutulduğu **durumların** (**state**) ve nesnelerin göstereceği **davranışlarının** (**behavior**) tanımlandığı bu plana **sınıf** (**class**) adı verilir.
2. **Nesne** (object): Bir mimari plandan birçok ev yapılabileceği gibi, bir sınıftan birçok nesne imal edilebilir. Sınıfta uygun olarak imal edilen her **nesne** (**object**) sınıfın bir **örneğidir** (**instance**). İmal edilen her bir nesnenin kendine özgü verisi bulunur. **Durumlar** (**state**), nesnelere ilişkin verilerin tutulduğu **alanlardır** (**field**).
3. **Soyutlama** (abstraction): Bir nesnenin veya bileşenin tasarımına ilişkin detay yerine ihtiyacımızı gördüğü kadar kullanılması işlemidir. Yani nesne ve bileşenleri **soyutlama** (**abstraction**) yaparak kullanmalıyız.
4. **Sarmalama** (encapsulation): Nesneye ilişkin durum ve davranışların nesne kontrolünde olmasıdır. Nesnenin verisine ve davranışlarına dışarıdan müdahale kontrollü biçimde olmalıdır.
5. **Miras Alma** (inheritance): Bir sınıfın başka sınıflardan nitelik ve davranışları devralmasıdır.
6. **Çok Biçimlilik** (polymorphism): Çalıştırma ortamında bulunan birden çok nesneye aynı mesajın verilip karşılığında her bir nesnenin farklı davranış göstermesi işlemidir.

Sınıf ve Nesne

Kısacası sınıflar, verileri tanımlar ve bu verileri işlemek için yöntemler sağlar. Bu yöntemler sınıfın verilerini işleyecek şekilde icra edilirler. Bu nedenle, sınıflar somut uygulama ayrıntılarını sınıf verilerinden ayırarak soyutlama sağlar.

Python dilinde nesne verilerini tutan **durumlar** (**state**), **nitelikler** (**attribute**) olarak adlandırılır. Nesnenin bu durumlarını işleyecek fonksiyonlar da **yöntem** (**method**) olarak adlandırılır. Sınıflarda nitelikler ve yöntemlerden oluşur. Nitelikler değişkenler olarak, yöntemler ise fonksiyonlar olarak tanımlanır. Aşağıdaki şekilde tanımlanır;

```
class sınıf-kimliği(object):  
    """Sınıfı tanımlayan docstring"""  
  
    #Sınıf Durumları/Niteklilikleri  
    sınıf-değişkeni = ilk-değer  
  
    def __init__(self): #Nesne Yapıcı  
        örnek-değişkeni=ilk-değer #Nesne Durumları/nitelikleri  
  
    #Davranışlar/Yöntemler  
    def davranış-kimliği(self, parametre):  
        """Yöntemi Tanımlayan docstring"""  
        parametre= alan-değişkeni-kimliği+parametre  
        return parametre
```

Bir nesne, belirli bir sınıfının örneğidir. Bir mimari plandan nasıl ki birden fazla ev imal edilebilir. Bir sınıftan da istenildiği kadar **örnek** (**instance**) alınabilir. Yani **nesne** (**object**) imal edilebilir.

Python dilinde sınıf üyelerine varsayılan olarak **halka açık (public)** olarak erişilir. Yani C++/Java/C# dillerindeki **public** anahtar kelimesinin davranışını taklit eder. Yani örnek ve sınıf niteliklerine dışardan erişim ve değişiklik her zaman mümkündür.

Durumlar nesnelere ve sınıflara ait verilerdir. Tüm sınıf içerisinde ve **imal edilecek nesneler tarafından ortak kullanılacak nitelikler önce tanımlanır** ve bu nitelikler **sınıf nitelikleri (class attribute)** olarak adlandırılır. Bu niteliklere sınıf dışından erişilebilir ve değiştirilebilir. Bu nitelikler varsayılan nesne verileri için ve **tekil nesne (singleton)** oluşturulması halinde yararlıdır.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik

kişi=Kişi() #Kişi sınıfından bir örnek (instance) oluşturuldu ve
            #kişi kimlikli değişken bu nesneye referans gösterildi.
            #Kısaca Kişi tipli kişi değişkeni tanımlandı.
Kişi.kişiSayısı +=1
print (type(kişi)) #<class '__main__.Kişi'>
print (kişi.__doc__) #Kişi Sınıfı
print (Kişi.kişiSayısı) #1
başkakişi=Kişi() #Kısaca Kişi tipli başkakişi değişkeni tanımlandı.
Kişi.kişiSayısı +=1
print (Kişi.kişiSayısı) #2
```

Sınıf tanımında yer alan her **yöntem (method)**, imal edilecek ilgili nesneyi birinci parametre olarak geçirir. Bunun için **self** kelimesi kullanılır. Bir ev planını değiştirmeden farklı renklerde ev imal edilebilir. Benzer şekilde bir sınıftan da farklı veriler içeren yani farklı **durumlarda (state)** nesneler imal edilebilir. Bunu sağlayan sınıftaki **özel bir yöntem (special method)** olan **__init__()** adlı nesne **yapıcısıdır (constructor)**. Yapıcı içerisinde tanımlanan nitelikler sadece imal edilen nesneye aittir. Bu niteliklere **örnek nitelikleri (instance attribute)** adı verilir.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik

    def __init__(self):
        """Kişi Sınıfı Yapıcısı"""
        self.adı='ilhan' #adı kimlikli örnek niteliği (instance attribute)
        self.yaşı=50      #yaşı kimlikli örnek niteliği (instance attribute)
        Kişi.kişiSayısı+=1 #her nesne imalatında 1 artar

kişi=Kişi() #Kişi sınıfından bir örnek (instance) oluşturuldu ve
            #kişi kimlikli değişken bu nesneye referans gösterildi.
            #Kısaca Kişi tipli kişi değişkeni tanımlandı.
print (type(kişi)) #<class '__main__.Kişi'>
print (kişi.__doc__) #Kişi Sınıfı
print (kişi.__init__.__doc__) #Kişi Sınıfı Yapıcısı
print (Kişi.kişiSayısı) #1 #Sınıf niteliğine erişim
print (kişi.adı) #ilhan #örneğin (kişi) niteliğine erişim
print (kişi.yaşı) #50    #örneğin (kişi) değişkenine erişim
başkakişi=Kişi() #Kısaca Kişi tipli başkakişi değişkeni tanımlandı.
print (Kişi.kişiSayısı) #2
print (başkakişi.adı) #ilhan
print (başkakişi.yaşı) #50
```

Örnek niteliğinde (instance attribute) yapılan değişiklikler **yalnızca tanımlandığı nesneyi etkiler!**

Her bir örnek değişkeni bellekten silineceği zaman ise sınıfın **__del__()** isimli özel yöntemi çağrılır. Bu yöntem **yıkıcı (destructor)** adı verilir.

```
class Kişi(object):
    """Kişi Sınıfı"""
```

```

KişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik

def __init__(self):
    """Kişi Sınıfı Yapıcısı"""
    self.adı='ilhan'
    self.yaşı=50
    Kişi.KişiSayısı+=1 #her nesne imalatında 1 artar

def __del__(self):
    """Kişi Sınıfı Yıkıcısı."""
    Kişi.KişiSayısı-=1 #her nesne silindiğinde 1 eksilir

kişi=Kişi()
print (Kişi.kişiSayısı) #1
başkakişi=Kişi() #Kısaca Kişi tipli başkakişi değişkeni tanımlandı.
print (Kişi.kişiSayısı) #2
del başkakişi
print (Kişi.kişiSayısı) #1

```

Özel yöntemlerden en çok kullanılan diğer iki yöntem `__repr__` ve `__str__` yöntemleridir. `__str__` imal edilen nesnenin metin olarak karşılığını veren yöntemdir. `__str__` kullanıcılar için kullanıcı dostu çıktı oluştururken, `__repr__` ideal olarak nesneyi yeniden imal edebilmek için ayrıntılı, geliştirici odaklı metin oluşturur.

```

class Araba:
    def __init__(self, marka, model):
        self.marka = marka
        self.model = model

    def __str__(self):
        # Kullanıcı için okunabilir
        return f"{self.marka} {self.model}"

    def __repr__(self):
        # Geliştirici için detaylı
        return f"Araba('{self.marka}', '{self.model}')"

```

Bir de her sınıfta ön tanımlı olarak `__doc__`, `__name__` ve `__module__` nitelikleri bulunur.

```

class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik
    def __init__(self):
        """Kişi Sınıfı Yapıcısı Parametre Almaz!."""
        Kişi.kişiSayısı+=1
        self.adı='ilhan' #örnek niteliği
        self.yaşı=50 #örnek niteliği
    def __del__(self):
        Kişi.kişiSayısı-=1

kişi=Kişi()
başkakişi=Kişi()
print(Kişi.__doc__) #Kişi Sınıfı
print(Kişi.__name__) #Kişi
print(Kişi.__module__) #__main__

```

Sınıflardan imal edilen nesnelerin kendilerine özgü **davranışları** (**behavior**) olabilir. Aşağıda bir `Kişi` sınıfı ve bu sınıftan imal edilen `kişi` ve `başkakişi` nesnelerine `adınıSöyle()` davranışını göstermesi için mesaj gönderilmesi örneğine yer verilmiştir.

```

class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik

```

```

def __init__(self):
    """Kişi Sınıfı Yapıcısısı."""
    Kişi.kişiSayısı+=1
    self.adı='İlhan' #örnek niteliği (instance attribute)
    self.yaşı=50 #örnek niteliği (instance attribute)
def __del__(self):
    """Kişi Sınıfı Yıkıcısısı."""
    Kişi.kişiSayısı-=1
def adınıSöyle(self):
    """Kişi Sınıfından İmal Edilecek Nesnelerin Adını Konsola Yazdırır"""
    print('Adım:{}'.format(self.adı))
def yaşınıGetir(self):
    """Kişi Sınıfından İmal Edilecek Nesnelerin Yaşını Döndürür"""
    return self.yaşı

kişi=Kişi()
print('Kişi Sayısı',Kişi.kişiSayısı) #Kişi Sayısı 1
kişi.adınıSöyle() #Adım:İlhan
kişi.adı='Ali' # kişi nesnesinin örnek niteliği (instance attribute) değiştiriliyor
kişi.adınıSöyle() #Adım:Ali
başkakişi=Kişi()
başkakişi.adınıSöyle() #Adım:İlhan

```

Örnekte imal edilen tüm nesnelerin `adı` niteliği 'İlhan', `yaşı` niteliği ise 50 olacaktır. Bu durumda yapıcı parametrelili tanımlanarak her seferinde başka nitelikler yani durumlarla nesne imal edilebilir.

```

class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik
    def __init__(self, adı='İlhan', yaşı=50):
        """Kişi Sınıfı Yapıcısı Parametre Almaz!."""
        Kişi.kişiSayısı+=1
        self.adı=adı #örnek niteliği
        self.yaşı=yaşı #örnek niteliği
    def __del__(self):
        Kişi.kişiSayısı-=1

    def adınıSöyle(self):
        """Kişi Sınıfından İmal Edilecek Nesnelere Adını Konsola Yazdırır"""
        print('Adım:{}'.format(self.adı))
    def yaşınıGetir(self):
        """Kişi Sınıfından İmal Edilecek Nesnelerin Yaşını Döndürür"""
        return self.yaşı

kişi=Kişi()
print('Kişi Sayısı',Kişi.kişiSayısı) #Kişi Sayısı 1
kişi.adınıSöyle() #Adım:İlhan
kişi.adı='Ali' # kişi nesnesinin örnek niteliği (instance attribute) değiştiriliyor
kişi.adınıSöyle() #Adım:Ali
başkakişi=Kişi('Mehmet')
print('Kişi Sayısı',Kişi.kişiSayısı) #Kişi Sayısı 2
başkakişi.adınıSöyle() #Adım:Mehmet
print(başkakişi.yaşınıGetir()) #50
birbaşkakişi=Kişi('Abdullah',40)
print('Kişi Sayısı',Kişi.kişiSayısı) #Kişi Sayısı 3
birbaşkakişi.adınıSöyle() #Adım:Abdullah
print(birbaşkakişi.yaşınıGetir()) #40

```

Bir başka özel yöntem ise `__str__()` yöntemidir. Aslında Java/C# dillerinde olan `toString()` yöntemi işlevini yerine getirir. Yani imal edilen nesnenin metin olarak karşılığının ne olduğu belirleyen yöntemdir.

```

class Kişi(object):

```

```

"""Kişi Sınıfı"""
kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik
def __init__(self, adı='İlhan', yaşı=50):
    """Kişi Sınıfı Yapıcısı Parametre Almaz!."""
    Kişi.kişiSayısı+=1
    self.adı=adı #örnek niteliği
    self.yaşı=yaşı #örnek niteliği
def __del__(self):
    Kişi.kişiSayısı-=1
def adınıSöyle(self):
    """Kişi Sınıfından İmal Edilecek Nesnelere Adını Konsola Yazdırır"""
    print('Adım:{}'.format(self.adı))
def yaşıyıGetir(self):
    return self.yaşı
def __str__(self):
    return 'Adı:{},Yaşı:{}'.format(self.adı,self.yaşı)

kişi=Kişi('ilhan',50)
kişi.adınıSöyle() #Adım:İlhan
metin=kişi.__str__()
print(metin) #Tür:İnsan,Adı:ilhan,Yaşı:50
print(kişi) #Tür:İnsan,Adı:ilhan,Yaşı:50 #dolaylı olarak __str__() çağrılır
başkakişi=Kişi('Mehmet',45)
print(başkakişi) #Tür:İnsan,Adı:Mehmet,Yaşı:45

```

Bunların dışında sınıfımızda `__fonksiyon__(self)` şeklinde yöntemler de tanımlayabiliriz. Bütün bu tür yöntemlere **sihirli yöntemler** (**magic method**) denir ve Python dilindeki sınıfların önemli bir parçasını oluşturur.

Yöntemler

Yöntemler (**method**), bir sınıfın nesnesine aittir ve nesne davranışlarını gösterecek belirli işlemleri gerçekleştirmek için kullanılır. Python metotlarını üç farklı kategoriye ayırabiliriz: sınıf yöntemi, örnek yöntemi ve statik yöntem.

Sınıf yöntemi (**class method**), sınıf nitelikleri gibi sınıfın örneğine değil, sınıfa bağlı bir metottur. Sınıfın bir örneği yerine, sınıfın kendisi üzerinde çağrılabilir. Sınıf yönteminin aksine, **örnek yöntemi** (**instance method**) bir nesnenin örneklerine yani imal edilmiş nesnelere erişebilir. Ayrıca, tüm nesneler için ortak olduğu için sınıf değişkenine de erişebilir.

Sınıf yöntemi yazmanın iki yolu vardır. Birincisi dahili `classmethod()` fonksiyonunu ile örnek yöntemi sınıf yöntemine çevrilir.

```

class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik
    def __init__(self):
        """Kişi Sınıfı Yapıcısı"""
        self.adı='ilhan'
        self.yaşı=50
        Kişi.kişiSayısı+=1 #her nesne imalatında 1 artar
    def __del__(self):
        """Kişi Sınıfı Yıkıcısı."""
        Kişi.kişiSayısı-=1 #her nesne silindiğinde 1 eksilir
    def kişiSayısınıGöster(self):
        print('Kişi Sayısı:',Kişi.kişiSayısı)
    NesneSayısı = classmethod(kişiSayısınıGöster)

kişi=Kişi()
Kişi.NesneSayısı() #1
başkakişi=Kişi() #Kısaca Kişi tipli başkakişi değişkeni tanımlandı.
Kişi.NesneSayısı() #2

```

```
del başkakişi
Kişi.NesneSayısı() #1
```

İkincisinde ise sınıf yöntemi başında `@classmethod` dekoratörü kullanılır.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik
    def __init__(self):
        """Kişi Sınıfı Yapıcısı"""
        self.adı='ilhan'
        self.yaşı=50
        Kişi.kişiSayısı+=1 #her nesne imalatında 1 artar
    def __del__(self):
        """Kişi Sınıfı Yıkıcısı."""
        Kişi.kişiSayısı-=1 #her nesne silindiğinde 1 eksilir
    @classmethod
    def KişiSayısınıGöster(cls): #cls geleneksel olarak kullanılır. Başka kimlik de olabilir.
        print('Kişi Sayısı:',cls.kişiSayısı)

Kişi=Kişi()
Kişi.KişiSayısınıGöster() #1
başkakişi=Kişi()
Kişi.KişiSayısınıGöster() #2
del başkakişi
Kişi.KişiSayısınıGöster() #1
```

Statik yöntemler (**static method**), çağırılması için herhangi bir sınıfın örneğine veya nesneye ihtiyaç duymayan yöntemlerdir. Sınıf yöntemine çok benzer ancak farkı, statik yöntemin `self` nesnesine veya `cls` sınıfına referans gibi zorunlu bir argümana sahip olmamasıdır. Statik bir yöntem;

- Sıklıkla tekrar kullanılan görevleri gerçekleştirmek için yardımcı bir fonksiyon olarak kullanılabilir.
- Sınıf adını kullanarak çağırırız. Bu sayede sınıftan imal edilmiş nesne örneklerine olan bağımlılık ortadan kalkar.
- Sınıf durumundan bağımsız olarak davranışı değişmeden kaldığı için her zaman öngörülebilir.
- İlerde göreceğimiz **geçersiz kılmayı** (**overriding**) önlemek kullanılır.

Statik yöntemi yazmanın iki yolu vardır. Birincisi dahili `staticmethod()` fonksiyonunu ile örnek yöntemi statik yöntemine çevrilir.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik
    def __init__(self):
        """Kişi Sınıfı Yapıcısı"""
        self.adı='ilhan'
        self.yaşı=50
        Kişi.kişiSayısı+=1 #her nesne imalatında 1 artar
    def __del__(self):
        """Kişi Sınıfı Yıkıcısı."""
        Kişi.kişiSayısı-=1 #her nesne silindiğinde 1 eksilir
    def KişiSayısınıGöster(): #parametrede self YOK!
        print('Kişi Sayısı:',Kişi.kişiSayısı)
    NesneSayısı = staticmethod(KişiSayısınıGöster)

Kişi.NesneSayısı() #0
Kişi=Kişi()
Kişi.NesneSayısı() #1
başkakişi=Kişi()
Kişi.NesneSayısı() #2
del başkakişi
Kişi.NesneSayısı() #1
```

İkincisinde ise sınıf yöntemi başında `@staticmethod` dekoratörü kullanılır.

```
class Kişi(object):
    """Kişi Sınıfı"""
    kişiSayısı=0 #sınıf niteliği (class attribute) tüm nesneler için ortak nitelik
    def __init__(self):
        """Kişi Sınıfı Yapıcısı"""
        self.adı='İlhan'
        self.yaşı=50
        Kişi.kişiSayısı+=1 #her nesne imalatında 1 artar
    def __del__(self):
        """Kişi Sınıfı Yıkıcısı."""
        Kişi.kişiSayısı-=1 #her nesne silindiğinde 1 eksilir
    @staticmethod
    def KişiSayısınıGöster(): #parametrede self YOK!
        print('Kişi Sayısı:',Kişi.kişiSayısı)

Kişi.KişiSayısınıGöster() #0
kişi=Kişi()
Kişi.KişiSayısınıGöster() #1
başkakişi=Kişi()
Kişi.KişiSayısınıGöster() #2
del başkakişi
Kişi.KişiSayısınıGöster() #1
```

Erişim Belirleyiciler

Erişim belirleyiciler (**access modifier**), sınıf üyelerine yani değişkenlere ve yöntemlere sınıf dışından erişimi kısıtlamak için kullanılır. **Halka açık** (**public**), **korumalı** (**protected**) ve **mahrem** (**private**) olmak üzere üç tür erişim belirleyici vardır.

Şimdiye kadar örneklerde tanımlanan sınıflarda nitelikler ve yöntemler varsayılan olarak halka açık olarak belirlenmiştir. Yani sınıf üyelerine sınıf ve nesne dışından erişilebilir.

Bazı durumlarda imal edilecek nesnelerin bazı üyelerine dışarıdan erişim kapatılmak istenir. Bu tür üyeler **mahrem** (**private**) üyeler olarak adlandırılır. Bir üyeyi mahrem yapmak için üye kimliği önünde iki altçizgi karakteri (`__`) kullanılır. Bazı üyelere de mevcut sınıf ve bu sınıftan miras alan sınıflar (ileride anlatılacaktır) dışında erişime kapatılır. Bu tür üyeler **korumalı** (**protected**) üye olarak adlandırılır. Korumalı bir üyenin kimliği önünde tek alt çizgi (`_`) karakteri kullanılır.

```
class Kişi(object):
    """Kişi Sınıfı"""
    def __init__(self, adı, sır, maaş):
        """Kişi Sınıfı Yapıcısı"""
        self.adı=adı #halka açık (public) nitelik
        self.__sır=sır #mahrem (private) nitelik
        self._maaş=maaş #korumalı(protected) nitelik
    def kişiBilgileri(self):
        print('Ad:{},Sır:{},Maaş{}'.format(self.adı,self.__sır,self._maaş))

kişi=Kişi('İlhan','Ben bişey biliyorum',20000)
print(kişi.adı) #İlhan
#print(kişi.__sır) #Hata: 'Kişi' object has no attribute '__sır'
print(kişi._maaş) #20000
kişi.kişiBilgileri() #Ad:İlhan,Sır:Ben bişey biliyorum,Maaş20000
```

Burada imal edilen `kişi` nesnesinin `kişiBilgileri()` yöntemi erişim belirleyicisi ne olursa olsun kişinin tüm verilerine eriştiği gözden kaçırılmamalıdır.

Soyutlama (Abstraction)

Soyutlama (**abstraction**), nesne yönelimli programlamanın önemli ilkelerinden biridir. Bu, bir nesneyle ilgili yalnızca önemli verilerin dış dünyaya açıldığı ve diğer tüm detayların gizlendiği bir programlama yaklaşımını ifade eder. Bu yaklaşım, karmaşıklığı azaltmaya ve uygulama geliştirme verimliliğini artırmaya yardımcı olur.

İki tür soyutlama vardır. Biri, nesnenin gizlendiği verilerle ilgili **veri soyutlamasıdır** (**data abstraction**); bu, gizli veriye nesne içinden erişilebilen bir veri yapısı aracılığıyla ulaşılır. Diğer tür ise süreç ya da **kontrol soyutlamasıdır** (**control abstraction**). Bu ise, nesnenin gerçekleştirdiği sürecin temel uygulama detaylarının gizlenmesini ifade eder.

Aşağıda verilen Kişi sınıfını göz önüne alalım. Bu sınıftan nesne imal edildikten sonra yaşını -1, 160, 2000, ... gibi anormal değerler vermemize kimse engel olamaz.

```
class Kişi(object):
    """Kişi Sınıfı"""
    def __init__(self,adı,yaş):
        """Kişi Sınıfı Yapıcısı"""
        self.adı=adı
        self.yaş=yaş
```

Bu gibi problemlerin üstesinden **veri soyutlaması** (**data abstraction**) ile gelebiliriz. Yani **yaş** alanını **mahrem** (**private**) yapıp bu alanın değerini kontrollü bir şekilde belirleyen **set** yöntemi yazabiliriz. Benzer şekilde bu alan mahrem olduğundan **get** yöntemi yazıp nesne verisini dışarıya aktarabiliriz.

```
class Kişi(object):
    """Kişi Sınıfı"""
    def __init__(self,adı='İlhan',yaş=50):
        """Kişi Sınıfı Yapıcısı"""
        self.adı=adı
        self.__yaş=yaş
    def setYaş(self,yaş):
        if yaş<0:
            print("Hata:Yaş değeri sıfırın altında olamaz!")
        elif yaş>120:
            print("Hata:Yaş değeri en fazla 120 olabilir!")
        else:
            self.__yaş=yaş
    def getYaş(self):
        return self.__yaş
```

```
ali=Kişi('Ali',40)
ali.setYaş(200) # Hata
print(ali.getYaş()) #40
```

Veri soyutlamasına benzer şekilde imal edilen nesnenin bazı davranışlarına dışarıdan erişim kontrollü bir şekilde yapılması gerekir. Bu işleme ise **kontrol soyutlaması** (**control abstraction**) adı verilir. Aşağıda **HesapMakinesi** sınıfı ve bu sınıftan imal edilen nesnelerin **ekranıyenile** davranışı için kontrol soyutlaması yapılmıştır.

```
class HesapMakinesi(object):
    """Hesap Makinesi Sınıfı"""
    def __init__(self):
        """Hesap Makinesi Sınıfı Yapıcısı"""
        self.__sonuç=0

    def __ekranıYenile(self): #hesap makinesi nesnesinin mahrem davranışı
        print('İşlem',self.__sonuç)

    def setHesapMakinesi(self,değer):
        self.__sonuç=değer
```



```

        self.__ekraniYenile()

    def makineyiSıfırla(self):
        self.__sonuç=0
        self.__ekraniYenile()

    def faktöriyelHesapla(self):
        temp=1
        for x in range(1,self.__sonuç+1):
            temp=temp*x
        self.__sonuç=temp
        self.__ekraniYenile()

makine=HesapMakinesi()
makine.setHesapMakinesi(10) #10
makine.makineyiSıfırla() #0
makine.setHesapMakinesi(5) #5
makine.faktöriyelHesapla() #120
#makine.__ekraniYenile() yazılamaz çünkü mahrem bir davranıştır.

```

Sarmalama (Encapsulation)

Bir sınıf üzerinde **veri soyutlaması** (**data abstraction**) veriler üzerine kılıf çekilir, hem de **kontrol soyutlaması** (**control abstraction**) ile davranışların üzerine bir kılıf çekilir. Bir sınıf üzerinde her iki soyutlamanın yapılmasına sarmalama (encapsulation) adı verilir. Soyutlamaların gerekli olup olmadığına programcı karar verir.

Sarmalama işlemi örneklerde görüldüğü üzere **erişim değiştiricilerle** (**access modifier**) yapılır. Böylece imal edilen nesnelerin verileri ve davranışları güvenli bir şekilde dış dünyaya açılmış olur. Böylece, sınıf ve nesne gibi bileşenlerin içeriğini bilmeden **soyut** (**abstract**) olarak kullanmak mümkün olmaktadır. Programcı tasarladığı sınıfın soyut olarak katlanılacağını düşünerek sınıfları tasarlamalıdır.

Özetle **sarmalama** (**encapsulation**) tekniğini uygulamak, kullanımı kolay **sınıf** (**class**), **bileşen** (**component**), **kütüphane** (**library**) ve **çerçeve yapılar** (**framework**) elde etmemizi sağlar;

- n adet **veri** (**data**) ve n adet **davranış** (**behavior**) sarmalama işlemine tabi tutulursa **sınıf**,
- n adet sınıf sarmalama işlemine tabi tutulursa **bileşen**,
- n adet bileşen sarmalama işlemine tabi tutulursa **kütüphane**,
- n adet kütüphane sarmalama işlemine tabi tutulursa **çerçeve yapılar** oluşur.

Böylece bize projemizde **proje süresini ve maliyetini kısaltma üstünlüğünü** verir.

Kalıtım (Inheritance)

Öğrenci, öğretmen ve müdürün olduğu bir sistemi örnek olarak ele alalım. Bunlardan her birinden nesne imal etmek için her birine ayrı ayrı sınıf tanımlamak gerekir.

```

class Öğrenci(object):
    def __init__(self):
        self.yaş=20
        self.cinsiyet='E'
        self.ogrenciNo=0
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    def dersÇalış(self):
        print('Ders Çalışıyorum')

öğrenci=Öğrenci()
öğrenci.dersÇalış()
öğrenci.cinsiyetiniSöyle()

```

Öğretmen için ayrı bir sınıf;

```
class Öğretmen(object):
    def __init__(self):
        self.yaş=50
        self.cinsiyet='E'
        self.verdiğiDersKodu='BIL101'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    def dersVer(self):
        print('Ders Veriyorum')
```

```
öğretmen=Öğretmen()
öğretmen.dersVer()
öğretmen.cinsiyetiniSöyle()
```

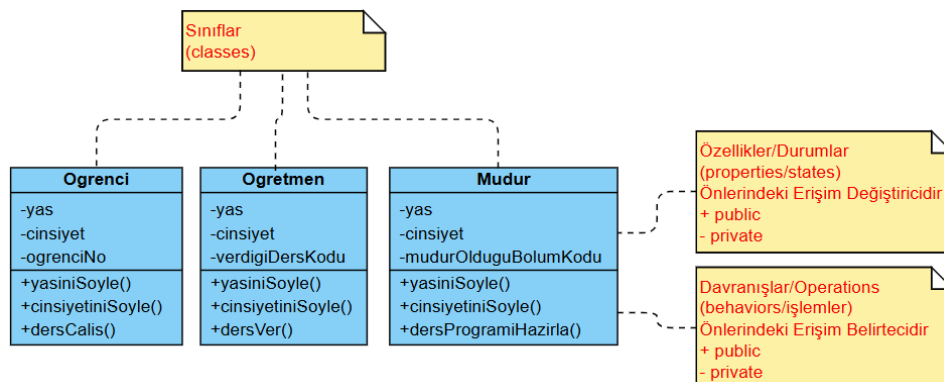
Müdür için ayrı bir sınıf;

```
class Müdür(object):
    def __init__(self):
        self.yaş=55
        self.cinsiyet='K'
        self.müdürOlduğuBölümKodu='Elektronik'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    def dersProgramıHazırla(self):
        print('Ders Programı Hazırlıyorum')
```

```
müdür=Müdür()
müdür.dersProgramıHazırla()
müdür.yaşınıSöyle()
```

Bu sınıfların her birinde `yaş` ve `cinsiyet` gibi ortak **özellikler** (property) veya `yaşınıSöyle()` ve `cinsiyetSöyle()` gibi ortak **davranışlar** (behavior) bulunur. Her sınıf için bunlar tekrar tekrar tanımlanırlar. Hâlbuki bu gibi ortak özellikler ve davranışlar tek bir sınıf altında toplanabilir. Bütün bu ortak özellik ve davranışlar tek bir sınıfta toplandığı zaman daha yönetilebilir bir kod elde edilir.

Yazılım analizi yapılırken takım içerisinde iletişimi kolaylaştırmak için aşağıdaki **sınıf diyagramları** (class diagram) kullanılır. Bunlar **Unified Modelling Language-UML** dilinin bir parçasıdır.



Şekil 21. Öğrenci, Öğretmen ve Müdür UML Sınıf Diyagramları

Ortak özellik ve davranışların bir sınıfta toplanarak bu sınıftan miras alınmasına kalıtım (**inheritance**) adı verilir. Yukarıdaki örnekte ortak özellik ve davranışlar **Kişi** sınıfına toplanarak aşağıdaki şekilde kod yeniden düzenlenebilir;

Aşağıda verilen ve Kişi olarak tanımlanan **genel sınıf** (**general class**), **taban sınıf** (**base class**) veya **ebeveyn sınıf** (**parent class**) olarak adlandırılır. Daha sonra tanımlanacak Öğrenci, Öğretmen ve Müdür sınıflarının ortak özellik ve davranışlarını barındırır.

```
class Kişi(object):
    def __init__(self):
        self.yaş=20
        self.cinsiyet='E'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
```

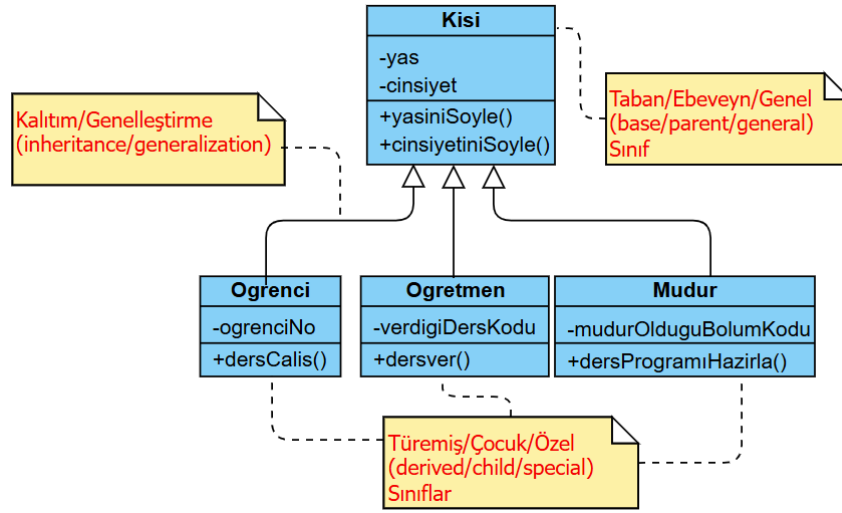
Aşağıda taban sınıftan **türemiş** (**derived**) Öğrenci, Öğretmen ve Müdür sınıfları verilmiştir. Bu sınıflar genel sınıf olan Kişi sınıfının yanında kendine özel özellik ve davranışlara da sahiptir. **Özel sınıf** (**private class**), **türemiş sınıf** (**derived class**) veya **çocuk sınıf** (**child class**) olarak adlandırılan bu sınıflar, mirasçı olup, kendilerinden imal edilmiş nesneler; hem taban sınıfın özellik ve davranışlarını, hem de kendi özellik ve davranışlarını sergilerler.

```
class Öğrenci(Kişi):
    def __init__(self):
        super().__init__() #miras alınan sınıfın yapıcısı çağrılıyor.
        self.ogrenciNo=0
    def dersÇalış(self):
        print('Ders Çalışıyorum')
class Öğretmen(Kişi):
    def __init__(self):
        super().__init__() #miras alınan sınıfın yapıcısı çağrılıyor.
        self.verdiğiDersKodu='BIL101'
    def dersVer(self):
        print('Ders Veriyorum')
class Müdür(Kişi):
    def __init__(self):
        super().__init__() #miras alınan sınıfın yapıcısı çağrılıyor.
        self.müdürOlduğuBölümKodu='Elektronik'
    def dersProgramıHazırla(self):
        print('Ders Programı Hazırlıyorum')
```

Miras alan sınıflardaki yapıcılarda taban sınıftaki nitelikler devralmak istenmiyorsa `super().__init__()` yöntemi kullanılmaz. **Kişi** sınıfından miras alan bu üç sınıftan da nesne imal edilmiş ve çeşitli davranışlar göstermesi için kendilerine **ileti gönderilmiştir** (**message-passing**).

```
öğrenci=Öğrenci()
öğrenci.dersÇalış()
öğrenci.cinsiyetiniSöyle()
öğretmen=Öğretmen()
öğretmen.dersVer()
öğretmen.cinsiyetiniSöyle()
müdür=Müdür()
müdür.dersProgramıHazırla()
müdür.yaşınıSöyle()
```

Yeni haliyle sınıf diyagramları aşağıda verilmiştir;



Şekil 22. Kişi Taban Sınıfı Eklenmiş Kalıtım UML Sınıf Diyagramı

Değişim yönetimini kolayca yapma üstünlüğünü veren kalıtımın (inheritance) iki önemli özelliği vardır; Paylaşılan bir **ortak kod (shared code)** vardır ve paylaşılan kodda yapılan **değişiklik anında alt sınıflara yansır (snap change)**.

Örneğimizden gidecek olursak, Kişi sınıfında yapılacak bir özellik veya davranış eklemesi anında bu sınıftan türemiş sınıflarda yapılmış olur. Burada önemli olan türemiş sınıfta yapıcıda **super()** yöntemi doğru kullanılmalıdır. Kötü kullanım aşağıdaki gibidir;

```

class Kopek(Hayvan):
    def __init__(self, isim):
        Hayvan.__init__(self, isim) # Kötü
  
```

İyi kullanım se aşağıdaki gibi olmalıdır;

```

class Kopek(Hayvan):
    def __init__(self, isim):
        super().__init__(isim) # İyi - Python way
  
```

Mantık olarak bir ev, birden fazla mimari plandan hazırlanmayacağından, **çoklu kalıtım (multiple inheritance)** sınıf tasarımında tercih edilmez. Sınıflar, birden fazla taban sınıftan miras alamayacak şekilde tasarlanır. Bu nedenle saf nesne yönelimli diller olan Java ve C# gibi dillerde çoklu kalıtıma izin verilmez. Python dilinde aşağıda belirtilen kalıtım türleri mevcuttur;

1. Tek Kalıtım: Bir alt sınıf, tek bir üst sınıftan kalıtım alır.
2. Çoklu Kalıtım: Bir alt sınıf, birden fazla üst sınıftan kalıtım alır.
3. Çok Düzeyli Kalıtım: Bir sınıf, başka bir sınıftan türetilen bir sınıftan türetilir.
4. Hiyerarşik Kalıtım: Birden fazla sınıf, tek bir üst sınıftan kalıtım alır.
5. Hibrit Kalıtım: Birden fazla kalıtım türünün birleşimidir.

Yöntem çözümleme sırası (method resolution order-MRO), sınıf hiyerarşisinde bir yöntemi aradığı sıradır. Çoklu kalıtım olduğunda, MRO hangi yöntemin gerçekten çağrıldığını anlamak için çok önemlidir. **Hata ayıklamada (debug)** ve kodu etkili bir şekilde okumaya yardımcı olur.

```

class A:
    def metod(self):
        print("A")

class B(A):
    def metod(self):
        print("B")

class C(A):
    def metod(self):
        print("C")
  
```

```
class D(B, C):
    pass

print(D.__mro__) # MRO kontrol etme
# (<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class 'object'>)
```

Aşırı Yükleme

Nesne yönelimli programlamada bir sınıf aynı kimliğe sahip ancak farklı parametrelere işlem yapan birden fazla yönteme sahip olabilir. Bu duruma **yöntemlerin aşırı yüklenmesi** (**method overloading**) adı verilir. Yöntemleri aşırı yüklemek için parametre sayısını, parametre veri tipini veya her ikisini birden değiştirmemiz gerekir.

Java, C++ ve C# gibi diğer nesne yönelimli programlama dillerinin aksine Python, varsayılan olarak yöntem aşırı yükleme özelliğini desteklemez. Ancak bunu yapmanın yolları vardır. Aşağıdaki kodda gösterildiği gibi bir metodu birden çok kez tanımlarsanız, son tanım öncekileri geçersiz kılar. Bu nedenle, Python'da bu şekilde metod aşırı yüklemesi yapmak hataya neden olur.

```
class Hesaplayıcı:
    def toplama(self, x, y):
        return x+y
    def toplama(self, a, b, c):
        return a+b+c

nesne = Hesaplayıcı()

print (nesne.toplama(10,20,30))
print (nesne.toplama(10,20)) # Hata! Toplama fonksiyonu 3 argüman bekler.
```

Yöntem aşırı yüklemesini simüle etmek için, yöntem argümanlarına varsayılan değeri **None** olarak tanımlayarak geçici bir çözüm kullanabiliriz, böylece yöntem bir, iki veya üç argümanla kullanılabilir.

```
class Hesaplayıcı:
    def toplama(self, a = None, b = None, c = None):
        x=0
        if a !=None and b != None and c != None:
            x = a+b+c
        elif a !=None and b != None and c == None:
            x = a+b
        return x

nesne = Hesaplayıcı()
print (nesne.toplama(10,20,30)) #60
print (nesne.toplama(10,20)) #90
```

Python standart kütüphanesinde yöntem aşırı yüklemesini uygulamak için başka bir düzenleme yoktur. Ancak, bu amaçla **MultipleDispatch** adlı üçüncü taraf bir modülden bir dağıtım işlevi kullanabiliriz. Bunun için bu modül projemize dahil etmeliyiz.

```
>>> pip install multipledispatch
```

Bu modülde bir **@dispatch** dekoratörü bulunur. Aşırı yüklenecek yönteme iletilecek argüman sayısını alır.

```
from multipledispatch import dispatch
class Hesaplayıcı:
    @dispatch(int, int)
    def toplama(self, x, y):
        return x+y
    @dispatch(int, int, int)
    def toplama(self, a, b, c):
        return a+b+c
```

```
nesne = Hesaplayıcı()
print (nesne.add(10,20,30))
print (nesne.add(10,20))
```

Çifte sevkiyat (**double dispatch**), bir sınıfı hiçbir şekilde değiştirmeden, söz konusu sınıfa etkin biçimde yöntem eklenebilmesi işlemidir. Bu çok bilinen bir tekniktir ve **Common Lisp Object System (CLOS)** gibi programlama dilleri tarafından desteklenir. C++, C#, Java gibi birçok nesne yönelimli programlama dili **tek sevkiyat** (**single dispatch**) yöntemi ile çalışır.

Geçersiz Kılma (Overriding)

Bir yöntemi **geçersiz kılma** (**overriding**), bir alt sınıfta, üst sınıfındaki bir yöntemle aynı adı taşıyan bir yöntem tanımlamayı ifade eder. Bu durumda, Python yorumlayıcısı, çalışma zamanında hangi yöntemin çağrılacağını, başvuru olan gerçek nesneye göre belirler.

Üst sınıf yöntemlerinizi her zaman geçersiz kılabilirsiniz. Üst sınıf yöntemlerini geçersiz kılmanın nedeni, alt sınıfınızda yöntemin farklı davranış göstermesini istememizdir.

```
class Taban:
    def yöntem(self):
        print ('Taban Sınıf Nesnesinin Yöntemi Çağrıldı.')

class Türemiş(Taban):
    def yöntem(self):
        print ('Türemiş Sınıf Nesnesinin Yöntemi Çağrıldı.')

tabannesne = Taban()
tabannesne.yöntem() #Taban Sınıf Nesnesinin Yöntemi Çağrıldı.
türemişnesne = Türemiş()
türemişnesne.yöntem() #Türemiş Sınıf Nesnesinin Yöntemi Çağrıldı.
```

Çok Biçimlilik (Polymorphism)

Çok biçimlilik (**polymorphism**) terimi, bir fonksiyonun veya yöntemin farklı bağlamlarda farklı şekillerde kullanılmasını ifade eder. Kısaca **çok biçimlilik** (**polymorphism**), birden çok nesnenin olduğu bir ortamda, verilen **aynı iletiye** (**message**) karşı, nesnelerin **farklı davranış** (**behavior**) göstermeleridir (**same message-different behavior**). **Temel sınıf** (**base class**) yöntemin çok biçimliliği sağlayan yöntem olması için, aynı yöntemin farklı **alt sınıflarda** (**derived class**) farklı iş mantıkları ile **geçersiz kılınması** (**override**) gerekir.

Python dilinde, **ördek tiplmesi** (**duck typing**), **yöntem geçersiz kılma** (**method overriding**), **işleç veya yöntem aşırı yükleme** (**method overloading**) uygulamaları ile çok biçimlilik yapılabilir.

Ördek tiplmesi, bir nesnenin türünün veya sınıfının, tanımladığı yöntemlerden daha az önemli olduğu bir kavramdır. Bu kavramı kullanarak, bir nesne üzerindeki herhangi bir yöntemi, türünü kontrol etmeden, yöntem mevcut olduğu sürece çağırabilirsiniz.

```
class Ordek:
    def seslen(self):
        return "Vak, Vak!"

class BaskaKus:
    def seslen(self):
        return "Civ Civ!"

def sesCikar(ordek):
    print(ordek.seslen())

duck = Ordek()
anotherBird = BaskaKus()
```

```
sesCikar(duck)    #Vak, Vak!
sesCikar(anotherBird) #Civ, Civ!
```

Geçersiz kılma ve aşırı yükle ile çok biçimlilik aşağıdaki şekilde yapılır;

```
class Kişi(object):
    def __init__(self):
        self.yaş=20
        self.cinsiyet='E'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    def raporYaz(self):
        print('Kişi Raporu: ...')

class Öğrenci(Kişi):
    def __init__(self):
        super().__init__()
        self.ogrenciNo=0
    def dersÇalış(self):
        print('Ders Çalışıyorum')
    def raporYaz(self):
        print('Öğrenci Raporu: ...')

class Öğretmen(Kişi):
    def __init__(self):
        super().__init__()
        self.verdiğiDersKodu='BIL101'
    def dersVer(self):
        print('Ders Veriyorum')
    def raporYaz(self):
        print('Öğretmen Raporu: ...')

class Müdür(Kişi):
    def __init__(self):
        super().__init__()
        self.müdürOlduğuBölümKodu='Elektronik'
    def dersProgramıHazırla(self):
        print('Ders Programı Hazırlıyorum')
    def raporYaz(self):
        print('Müdür Raporu: ...')

kişiler = [Öğrenci(), Öğretmen(),Müdür()]
for kişi in kişiler:
    kişi.raporYaz() #Her nesneye aynı mesaj veriliyor! Ama farklı davranış gösteriyorlar.
```

Yukarıdaki kod çalıştırıldığında aşağıdaki çıktı elde edilir;

```
Öğrenci Raporu: ...
Öğretmen Raporu: ...
Müdür Raporu: ...
```

Çok biçimlilik bize **çalıştırma anında** (run time) üstünlük sağlar. Bu nedenle nesneler de çalıştırma anında imal edilmiştir. Program çalıştırıldığında her **kişi** nesnesine aynı ileti verilmiş ama bu nesneler farklı davranış göstermiştir. Nesne yönelimli programlamada **dinamik bağlama** (dynamic binding) kavramı, çok biçimlilikle yakından ilişkilidir. Python dilinde dinamik bağlama, bir yöntemi veya niteliği derleme zamanı yerine çalışma zamanında çözümleme sürecidir.

Soyut Sınıf (Abstract Class)

Nesne yönelimli programlama terminolojisinde, bir sınıfın örneği/nesnesi oluşturulamıyorsa, bu sınıfa **soyut sınıf** (**abstract class**) denir. Soyut sınıfları diğer sınıfları oluşturmak için temel veya üst sınıf olarak kullanabilirsiniz.

Python dilinde soyut bir sınıf oluşturmak için, **ABC** modülünde tanımlanan **ABC** sınıfını miras alması gerekir. Bu modül, Python dilinin standart kütüphanesinde mevcuttur. Ayrıca, sınıfın en az bir **soyut yöntem** (**abstract method**) olmalıdır. Bu soyut yöntem, çağrılmayan ancak geçersiz kılınabilen yöntemdir. Soyut yöntem, `@abstractmethod` dekoratörüyle süslenir.

```
from abc import ABC, abstractmethod
class Kişi(ABC):
    def __init__(self):
        self.yaş=20
        self.cinsiyet='E'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    @abstractmethod
    def raporYaz(self):
        print('Soyut Yöntem')

# Kişi=Kişi() #Hata: Soyut Sınıftan Nesne İmal Edilemez!
```

Türemiş sınıfta sanal yöntemin geçersiz kılınması aşağıdaki şekilde yapılabilir;

```
from abc import ABC, abstractmethod
class Kişi(ABC):
    def __init__(self):
        self.yaş=20
        self.cinsiyet='E'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    @abstractmethod
    def raporYaz(self):
        print('Soyut Yöntem')

class Öğrenci(Kişi):
    def __init__(self):
        super().__init__()
        self.ogrenciNo=0
    def dersÇalış(self):
        print('Ders Çalışıyorum')
    def raporYaz(self):
        super().raporYaz() #istenirse soyut yöntem çağrılmayabilir.
        print('Öğrenci Raporu: ...')

öğrenci=Öğrenci()
öğrenci.raporYaz()
```

Yukarıdaki kod çalıştırıldığında aşağıdaki çıktı elde edilir;

```
Soyut Yöntem
Öğrenci Raporu: ...
```

Ara Yüzler yani Roller (Interface)

Ara yüzler (interface) nesnelerin sahip oldukları rollerdir (role). Roller bir grup davranışı (behavior) barındırır. Saf nesne yönelimli dillerde ara yüzler interface saklı kelimesiyle tanımlanırlar. Python dilinde bu anahtar kelime bulunmadığından roller, sınıf (class) olarak kodlanır.

Örnek olarak bir öğretmen; Evde baba rolünde olup babanın sahip olduğu çocuklara bakma, yemek yapma gibi davranışları gösterir. İşte öğretmen rolündedir ve öğrenmenin sahip olduğu; öğretme, sınav/değerlendirme yapma, karne hazırlama, ders notu hazırlama gibi davranışları vardır. Bu öğretmen müzisyen rolünde ise gitar çalma, akort yapma gibi davranışları da vardır. İşte bunların hepsi ayrı bir roldür ve her bir rolde o role ilişkin davranışları sergiler.

Bir sınıfa, bir veya birden fazla ara yüz uygulanabilir (interface implementation). Python dilinde interface kelimesi olmadığından ara yüzler soyut yöntemler içeren soyut sınıflar olarak tasarlanır. Bir sınıfın bir arayüzü miras almasına ara yüz gerçekleştirme (interface realization) adı da verilir.

Python dili, saf nesne yönelimli dillerin (Java, C# gibi) aksine çoklu kalıtımı (multiple inheritance) destekler, çünkü interface gibi bir anahtar kelimeye sahip olmadığından ara yüz olarak tanımlanmış birçok sınıftan miras alması gerekebilir.

Aşağıda rollerin soyut sınıflar üzerinden resmi (formal) tanımlaması yapılmıştır.

```
from abc import ABC, abstractmethod
class IBaba(ABC):
    @abstractmethod
    def cocugaBak(self):
        return
    @abstractmethod
    def HanımaYardımEt(self):
        return
class IMuzisyen(ABC):
    @abstractmethod
    def gitarCal(self):
        return
    @abstractmethod
    def piyanoCal(self):
        return
class IOgretmen(ABC):
    @abstractmethod
    def ogret(self):
        return
    @abstractmethod
    def dersNotuHazirla(self):
        return

class Kisi(IBaba, IMuzisyen, IOgretmen): # arayüz gerçekleştirme.
    def __init__(self):
        self.yaş=20
        self.cinsiyet='E'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    #IBaba Interface Yöntemleri
    def cocugaBak(self):
        print('Çocuğa Bakıyorum...')
    def HanımaYardımEt(self):
        print('Hanıma Yardım Ediyorum...')
    #IMuzisyen Interface Yöntemleri
    def gitarCal(self):
        print('Gitar Çalıyorum...')
```

```

def piyanoCal(self):
    print('Piyano Çalıyorum...')
#IOgretmen Interface Yöntemleri
def ogret(self):
    print('Öğretiyorum...')
def dersNotuHazirla(self):
    print('Ders Notu Hazırlıyorum...')

kisi=Kisi()
kisi.cocugaBak() #IBaba Rolü davranışı
kisi.gitarCal() #IMuzisyen Rolü davranışı
kisi.dersNotuHazirla() #IOgretmen rolü davranışı

```

Aynı örneğin **gayri resmi** (informal) tanımlaması aşağıda verilmiştir.

```

class IBaba(object):
    def cocugaBak(self):
        pass
    def HanımaYardımEt(self):
        pass
class IMuzisyen(object):
    def gitarCal(self):
        pass
    def piyanoCal(self):
        pass
class IOgretmen(object):
    def ogret(self):
        pass
    def dersNotuHazirla(self):
        pass

class Kisi(IBaba, IMuzisyen, IOgretmen):
    def __init__(self):
        self.yaş=20
        self.cinsiyet='E'
    def yaşınıSöyle(self):
        print('Yaşım:',self.yaş)
    def cinsiyetiniSöyle(self):
        print('Cinsiyetim:',self.cinsiyet)
    #IBaba Interface Yöntemleri
    def cocugaBak(self):
        print('Çocuğa Bakıyorum...')
    def HanımaYardımEt(self):
        print('Hanıma Yardım Ediyorum...')
    #IMusizyen Interface Yöntemleri
    def gitarCal(self):
        print('Gitar Çalıyorum...')
    def piyanoCal(self):
        print('Piyano Çalıyorum...')
    #IOgretmen Interface Yöntemleri
    def ogret(self):
        print('Öğretiyorum...')
    def dersNotuHazirla(self):
        print('Ders Notu Hazırlıyorum...')

kisi=Kisi()
kisi.cocugaBak() #IBaba Rolü davranışı
kisi.gitarCal() #IMuzisyen Rolü davranışı
kisi.dersNotuHazirla() #IOgretmen rolü davranışı

```

İç Sınıf (Inner Class)

Python dilinde bir sınıfın içinde tanımlanan sınıfa **iç sınıf** (**inner class**) denir. İç sınıfa **iç içe sınıf** (**nested class**) adı da verilir. İç sınıftan bir nesne imal edilirse yani örneklendirildiğinde bu nesne, ana sınıf tarafından da kullanılabilir. İç sınıf nesnesi, dış sınıfın **niteliklerinden** (**attribute**) biri haline gelir. İç sınıf, **kalıtımı** (**inheritance**) kodlamadan dış sınıfın niteliklerini otomatik olarak devralır. Bir sınıfta **birden fazla iç sınıf** (**multiple inner class**) tanımlanabileceği gibi **istenildiği kadar iç içe sınıf** (**multilevel inner class**) tanımlanabilir.

```
class Sirket: #outer class
    def __init__(self):
        self.Bolum1 = self.Muhasebe()
        self.Bolum2 = self.Pazarlama()
    def SirketAdiniYaz(self):
        print("Şirket Adı: Python A.Ş")

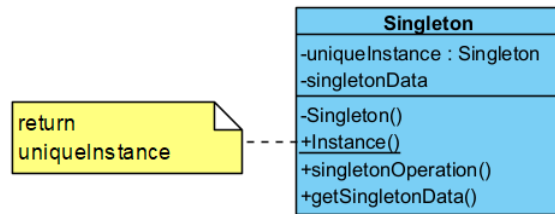
    class Muhasebe: #inner class
        def BolumunAdiniYaz(self):
            print("Bölüm Adı: Muhasebe")

    class Pazarlama: #inner class
        def BolumunAdiniYaz(self):
            print("Bölüm Adı: Pazarlama")

disNesne = Sirket()
disNesne.SirketAdiniYaz() # Şirket Adı: Python A.Ş
icNesne1 = disNesne.Bolum1
icNesne1.BolumunAdiniYaz() # Bölüm Adı: Muhasebe
icNesne2 = disNesne.Bolum2
icNesne2.BolumunAdiniYaz() # Bölüm Adı: Pazarlama
```

Tekil Nesne Sınıfı (Singleton)

Tekil nesne sınıfı (**singleton class**), singleton **tasarım deseninin** (**design pattern**) uygulanmasıdır; bu da sınıfın yalnızca bir nesneye sahip olabileceği anlamına gelir. Yoğun bir işlem gerçekleştirdiğinizde bellek kullanımını optimize etmenize yardımcı olur.



Şekil 23. Tekil Deseni UML Diyagramı

Python dilinde `__new__` veya `__init__` yöntemlerinin birini kullanarak tekil nesne sınıfı oluşturabiliriz. Aşağıda `__init__` yöntemiyle sınıfın kodlaması gösterilmiştir;

```
class Singleton:
    __tekilnesne = None # mahrem (private) nesne referansı

    @staticmethod
    def nesneImalEt():
        if Singleton.__tekilnesne == None:
            Singleton()
        return Singleton.__tekilnesne

    def __init__(self):
        if Singleton.__tekilnesne != None:
            raise Exception("Object exist!")
```

```

    else:
        Singleton.__tekilnesne = self

nesne1 = Singleton.nesneImalEt()
nesne2 = Singleton.nesneImalEt()
if nesne2 is nesne1:
    print("İki Nesne AYNIDIR!")

```

Alternatif olarak `__new__` yöntemi kullanılarak tekil nesne sınıfı kodlanabilir.

```

class SingletonClass:
    __tekilNesne = None #mahrem (private) tekil nesne referansı

    def __new__(cls):
        if cls.__tekilNesne is None:
            cls.__tekilNesne = super(SingletonClass, cls).__new__(cls)
        return cls.__tekilNesne

nesne1 = SingletonClass()
nesne2 = SingletonClass()
if nesne2 is nesne1:
    print("İki Nesne AYNIDIR!")

```

Yansıma (Reflection)

Nesne yönelimli programlamada **yansıma** (**reflection**), imal edilmiş bir nesnenin tipi, nitelikleri, davranışları, bir başka sınıfın alt sınıfı olup olmadığı gibi birçok konuda bilgisini çıkarma yeteneğidir. Python dilinin sağladığı standart kütüphane, bir nesnenin farklı özelliklerini yansıtan çeşitli işlevlere sahiptir. Yansımaya bazen **iç gözlem** (**introspect**) de denir. Aşağıda bu fonksiyonlar verilmiştir;

type() fonksiyonu

Bir nesnenin hangi sınıfa ait olduğunu söyler.

```

print (type(100)) #<class 'int'>
print (type(7.5)) #<class 'float'>
print (type(5+2j)) #<class 'complex'>
print (type("İlhan")) #<class 'str'>
print (type([10,20,30])) #<class 'list'>
print (type((10,20,30))) #<class 'tuple'>
print (type({'1':'bir', 2:'iki'})) #<class 'dict'>

class Sinif:
    pass
nesne=Sinif()
print(type(nesne)) #<class '__main__.Sinif'>

```

isinstance() fonksiyonu

Bir nesnenin verilen sınıfın bir örneği olup olmadığını yani verilen sınıftan imal edilip edilmediğini belirler. Aşağıdaki kod örneğinde bu fonksiyon `True` değerini geri döndürür.

```

print (isinstance(100, int))
print (isinstance(7.5, float))
print (isinstance(5+2j, complex))
print (isinstance("İlhan", str))
print (isinstance([10,20,30], list))
print (isinstance((10,20,30), tuple))
print (isinstance({'1':'bir', 2:'iki'}, dict))

class Sinif:
    pass
nesne=Sinif()

```

```
print(isinstance(nesne, Sinif))
```

issubclass() fonksiyonu

Bu fonksiyon bir sınıfın başka bir sınıfın alt sınıfı olup olmadığını kontrol eder. Sınıflar için kullanılır. Tüm Python sınıfları `object` sınıfından alt sınıflara ayrılır. Dolayısıyla, aşağıdaki `print` ifadelerinin çıktısı hepsi için `True` olacaktır.

```
class Sinif:
    pass

class AltSinif(Sinif):
    pass

print (issubclass(int, object))
print (issubclass(str, object))
print (issubclass(Sinif, object))
print (issubclass(AltSinif, object))
print (issubclass(AltSinif, Sinif))
```

callable() fonksiyonu

Bir nesne, belirli bir davranışı çağırıyorsa **çağrılabilir** (**callable**). Belirli bir davranışı gerçekleştiren bir Python fonksiyonu ise çağrılabilir bir nesnedir. Herhangi bir fonksiyon, **dahili** (**built-in**), **kullanıcı tanımlı** (**user-defined**) veya **yöntem** (**method**) çağrılabilir. `int`, `str` ve benzeri gibi dahili veri tiplerine sahip nesneler çağrılmaz. Bir sınıf örneği, `__call__()` y sahipse çağrılabilir.

```
def yontem():
    pass

print (callable("Hello")) #False
print (callable(12)) #False
print (callable(min)) #True
print (callable([1,2])) #False
print (callable(list.clear([1,2]))) #False
print (callable(yontem)) #True

class Sinif:
    def __init__(self):
        pass
    def __call__(self):
        print ("Bu Sınıftan İmal Edilen Nesneler Çağrılabilir!")

nesne = Sinif()
print ("callable?", callable(nesne)) #True
```

getattr() fonksiyonu

Bu fonksiyon, bir nesnenin adlandırılmış niteliğinin değerini almak için kullanılır.

```
class Kisi:
    def __init__(self):
        self.Adi = "İlhan"
        self.Soyadi="Özkan"
        self.Yasi=50

nesne = Kisi()
print (getattr(nesne, "Adi")) #İlhan
print (getattr(nesne, "Soyadi")) #Özkan
print (getattr(nesne, "Yasi")) #50
```

setattr() fonksiyonu

Bu fonksiyon, bir nesneye yeni bir **nitelik** (**attribute**) ekler ve ona bir değer atar. Ayrıca mevcut bir niteliğin değerini de değiştirebilir.

```
class Kisi:
    def __init__(self):
        self.Adi = "İlhan"

nesne = Kisi()
print (getattr(nesne, "Adi")) #İlhan

setattr(nesne, "Adi", "Mehmet")
setattr(nesne, "Soyadi", "Özkan")
setattr(nesne, "Yasi", 50)
print (nesne.Adi) #Mehmet
print (nesne.Soyadi) #Özkan
print (nesne.Yasi) #50
```

hasattr() fonksiyonu

Bu fonksiyon, nesnenin verilen niteliğe sahip olup olmadığını belirler. Sahipse **True** değerini, aksi durumda **False** değerini döndürür.

```
class Kisi:
    def __init__(self):
        self.Adi = "İlhan"
        self.Soyadi="Özkan"
        self.Yasi=50

nesne = Kisi()
print (hasattr(nesne, "Adi")) #True
print (hasattr(nesne, "Cinsiyeti")) #False
print (hasattr(nesne, "Kilosu")) #False
```

dir() fonksiyonu

Bu yerleşik fonksiyon argüman olmadan çağrılırsa, geçerli kapsamdaki adları döndürür. Herhangi bir nesne argüman olarak kullanıldığında, belirtilen nesnenin ve bu nesneden erişilebilen niteliklerin bir listesini döndürür.

```
print("dir(int):",dir(int))
print("dir(list):",dir(list))

class Kisi:
    def __init__(self):
        self.Adi = "İlhan"
        self.Soyadi="Özkan"
        self.Yasi=50
    def AdivSoyadiEkranaYaz(self):
        print("Adı:",self.Adi)
        print("Soyadı:", self.Soyadi)
    def YasiEkranaYaz(self):
        print("Yaşı:",self.Yas)

print("dir(Kisi):",dir(Kisi))
```

Kod çalıştırıldığında aşağıdaki çıktı elde edilir.

```
dir(int): ['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__',
 '__delattr__', '__dir__', '__divmod__', '__doc__', '__eq__', '__float__', '__floor__',
 '__floordiv__', '__format__', '__ge__', '__getattr__', '__getnewargs__', '__getstate__',
 '__gt__', '__hash__', '__index__', '__init__', '__init_subclass__', '__int__', '__invert__',
```

```
'__le__', '__lshift__', '__lt__', '__mod__', '__mul__', '__ne__', '__neg__', '__new__',
'__or__', '__pos__', '__pow__', '__radd__', '__rand__', '__rdivmod__', '__reduce__',
'__reduce_ex__', '__repr__', '__rfloordiv__', '__rlshift__', '__rmod__', '__rmul__',
'__ror__', '__round__', '__rpow__', '__rrshift__', '__rshift__', '__rsub__', '__rtruediv__',
'__rxor__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__',
'__truediv__', '__trunc__', '__xor__', 'as_integer_ratio', 'bit_count', 'bit_length',
'conjugate', 'denominator', 'from_bytes', 'imag', 'is_integer', 'numerator', 'real',
'to_bytes']
dir(list): ['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__',
'__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__',
'__getitem__', '__getstate__', '__gt__', '__hash__', '__iadd__', '__imul__', '__init__',
'__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__',
'__new__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__',
'__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'append', 'clear',
'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
dir(Kisi): ['AdiverSoyadiEkranayaYaz', 'YasiEkranayaYaz', '__class__', '__delattr__', '__dict__',
'__dir__', '__doc__', '__eq__', '__firstlineno__', '__format__', '__ge__', '__getattribute__',
'__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__',
'__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__',
'__sizeof__', '__static_attributes__', '__str__', '__subclasshook__', '__weakref__']
```

Dekoratorlar

Yazılım mühendisliğinde "Open/Closed" prensibi, bir nesnenin geliştirilmeye açık ama değiştirilmeye kapalı olması gerektiğini savunur. Python'da bir fonksiyonun veya sınıfın davranışını, onun kaynak koduna dokunmadan değiştirmemizi veya genişletmemizi sağlayan yapıya **dekoratör** (**decorator**) denir. Dekoratorlar, aslında başka bir fonksiyonu parametre olarak alan ve yeni bir fonksiyon döndüren **yüksek seviyeli fonksiyonlardır** (**higher-order functions**).

Kısaca **dekoratörler** (**decorator**), fonksiyonların veya yöntemlerin kodlarını değiştirmeden davranışlarını değiştirmenin veya genişletmenin esnek bir yoludur.

Fonksiyonların Nesne Olarak Kullanılması

Dekorator mantığını anlamak için Python'da fonksiyonların da birer "nesne" (first-class object) olduğunu hatırlamak gerekir. Fonksiyonlar; değişkenlere atanabilir, başka fonksiyonlara parametre olarak gönderilebilir veya bir fonksiyonun içinden geri döndürülebilir.

Dekorator Söz Dizimi ve @ Operatörü

Python'da bir fonksiyonu dekore etmek için `@dekorator_adi` ifadesi kullanılır. Bu, kodun okunabilirliğini artıran bir sentaktik şekerdir (syntactic sugar).

Temel Bir Dekorator Tasarımı

Aşağıdaki örnekte, bir fonksiyonun ne kadar sürede çalıştığını ölçen bir mühendislik aracı tasarlayalım:

```
import time
def sure_olcer(fonksiyon):
    def sarici(*args, **kwargs):
        baslangic = time.time()
        sonuc = fonksiyon(*args, **kwargs) # Asıl fonksiyon çalıştırılıyor
        bitis = time.time()
        print(f"{fonksiyon.__name__} çalışma süresi: {bitis - baslangic} sn.")
        return sonuc
    return sarici

@sure_olcer
def veri_isle(n):
    # Ağır bir hesaplama simülasyonu
    return sum(range(n))
```



```
# Kullanım
veri_isle(100000000)
```

Parametrelili Dekoratorler ve *args, **kwargs

Bir dekoratörün her türlü fonksiyona uyum sağlayabilmesi için (farklı sayıda parametre alan fonksiyonlar gibi), sarmalayıcı (wrapper) fonksiyonun esnek bir yapıda olması gerekir. Bu noktada kitabımızın 6. bölümünde işlediğimiz *args ve **kwargs yapıları devreye girer.

@functools.wraps Kullanımı

Bir fonksiyonu dekore ettiğinizde, o fonksiyonun teknik kimliği (ismi, dokümantasyon dizini vb.) dekoratörün içindeki sarmalayıcı fonksiyonla yer değiştirir. Bu durum, hata ayıklama (debugging) süreçlerinde kafa karışıklığına yol açabilir. Bu mühendislik zafiyetini önlemek için functools.wraps kullanılır.

```
from functools import wraps
def log_kaydi(f):
    @wraps(f)
    def wrapper(*args, **kwargs):
        print(f"{f.__name__} fonksiyonu çağrıldı.")
        return f(*args, **kwargs)
    return wrapper
```

Neden Dekoratör Kullanırsınız?

- Yetkilendirme (authorization): Kullanıcının bir fonksiyonu çalıştırma yetkisi olup olmadığını kontrol etmek.
- Loglama (logging): Fonksiyon çağrılarını ve çıktılarını merkezi bir dosyada kayıt altına almak.
- Önbelleğe Alma (caching): Ağır hesaplamaların sonuçlarını belleğe alarak performansı artırmak (@lru_cache).
- Hata Yönetimi: Fonksiyonun belirli hata durumlarında otomatik olarak yeniden denenmesini (retry) sağlamak.

Dekoratorler, kodun içine enine kesen ilgiler (cross-cutting concerns) eklemek için mükemmeldir. Ana iş mantığınızı (business logic) kirlletmeden, yardımcı işlemleri bu yapılarla yönetebilirsiniz.